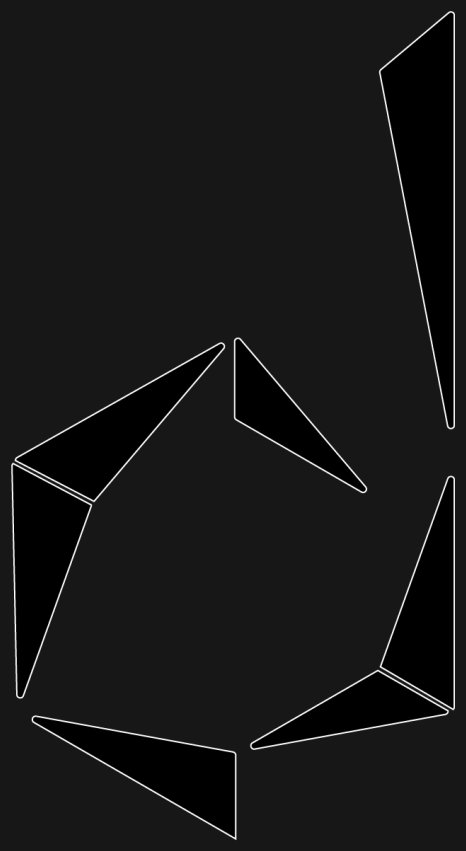
INGENIERÍA MECATRÓNICA



DI_CERO

DIEGO CERVANTES RODRÍGUEZ

PROGRAMACIÓN: DESARROLLO BACKEND

PYTHON

Introducción a Django

Contenido

Introducción a Django 2
Arquitectura MVT (Model View Template) 4
Model: ORM (Object-Relational Mapping) y Migraciones 7
Referencias 7

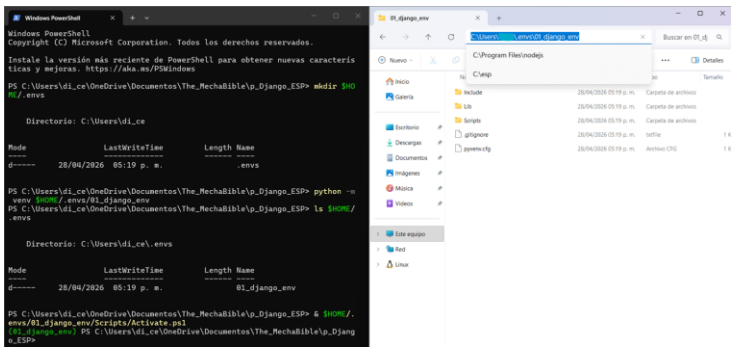
Introducción a Django

Django es un framework orientado a objetos (POO) para desarrollo web escrito en Python, inicialmente fue construido como un generador de blogs, pero poco a poco fue ganando popularidad y ahora es utilizado para construir aplicaciones, más que nada por su facilidad de uso, debido a que nos permite construir apps desde cero a producción en tiempo record. Inicialmente Instagram y Spotify crearon la primera versión de sus aplicaciones en Django.

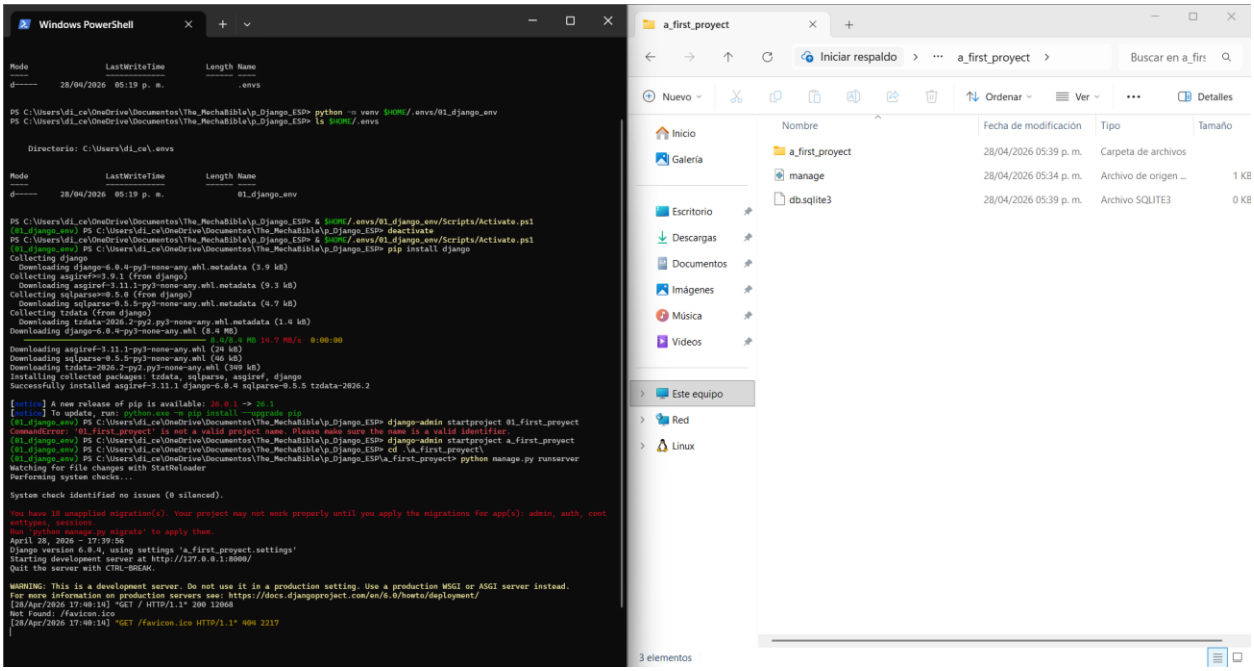
Django utiliza entornos virtuales, estos se refieren a paquetes que se puedan utilizar en diferentes versiones del sistema operativo de nuestra computadora, sin que se causen colisiones, temas de compilación o versionamiento del código.

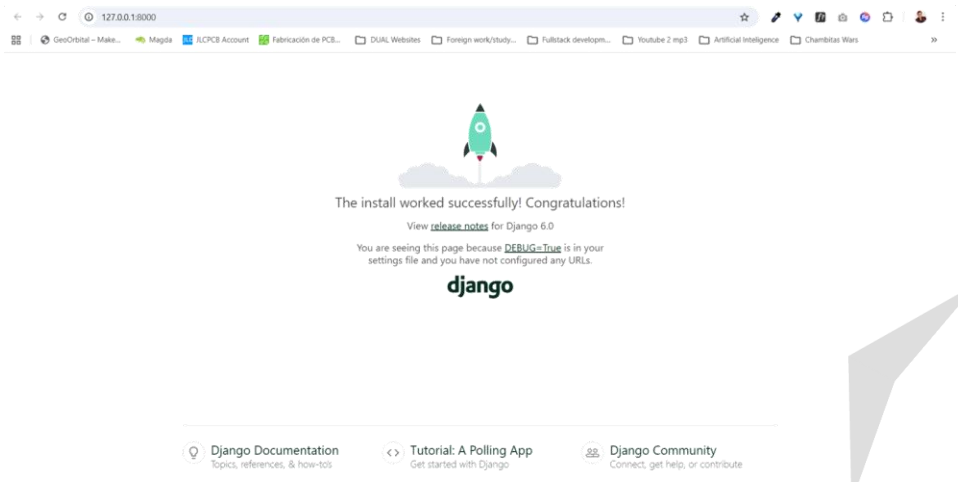
Al instalar Django debemos ejecutar el siguiente comando en nuestra terminal CDM o PowerShell, para ello ya debemos haber destinado una carpeta de Python donde se creará el entorno virtual, realizando su instalación y activación de la siguiente forma:

- **mkdir** *$HOME/.envs*: En PowerShell a veces conviene crear primero manualmente la carpeta que almacenará los entornos virtuales.
    - El entorno virtual se crea de esta forma si se quiere utilizar **el sistema operativo de Windows en vez de la WSL (Windows Subsystem for Linux)**, que es una opción donde se usa Linux dentro de Windows como el sistema operativo que guarda el entorno virtual.
- **python -m venv** *$HOME/.envs/01_django_env*: El comando principal para **crear el entorno virtual** es **python -m venv** y después se debe pasar como parámetro el *path* de donde queremos que se guarde este entorno virtual, en este caso parte de nuestra carpeta home C: al utilizar el comando *$HOME* y luego indicamos *el nombre de las carpetas posteriores que se crearán para almacenar el entorno virtual*.
    - Una vez que hayamos creado el entorno virtual dentro de la carpeta *~/.envs/*, con el comando **ls** podremos ver la carpeta del entorno virtual *01_django_env* creada con ese nombre dentro de ella.
- **&** *$HOME/.envs/01_django_env***/Scripts/Activate.ps1**: El comando principal para **activar el entorno virtual** es **&** *path***/Scripts/Activate.ps1**, al hacerlo correctamente, el nombre del entorno virtual aparecerá hasta el inicio de nuestra consola CMD o PowerShell entre paréntesis y cuando esto pase, todos los comandos que uilice después se ejecutarán en el entorno virtual, para que después pueda crear una aplicación que sea ejecutable en cualquier versión de mi sistema operativo.
    - Para salir del entorno virtual, debemos ejecutar el comando: **deactivate**



- Ya que tengamos el entorno virtual activado debemos utilizar el instalador de paquetes de Python **pip** para realizar la instalación de Django dentro de nuestro entorno de desarrollo activado, esto se realizará con el comando: **pip install django**
- **django-admin startproject** *a_first_proyect*: Una vez que instalamos Django, para crear nuestro proyecto debemos utilizar el comando **django-admin** y su método **startproject**, luego le asignaremos un *nombre*, para ello debemos evitar usar números al inicio y guiones medios (-) y en vez de eso hay que utilizar guiones bajos (_).
    - **cd** *a_first_proyect*: Luego deberemos ingresar a la reciente carpeta creada de nuestro proyecto y ahí deberemos ejecutar un archivo para que podamos correrlo.
    - **python manage.py runserver**: Al ejecutar correctamente los comandos de creación del proyecto Django, se creará el archivo **manage.py** en nuestra carpeta actual, este lo debemos ejecutar y entrar en nuestro localhost dentro del navegador (http://127.0.0.1:8000/) para ver si la página principal de Django ha sido lanzada, si esto se ha realizado correctamente, nuestro sitio ha sido configurado y creado.



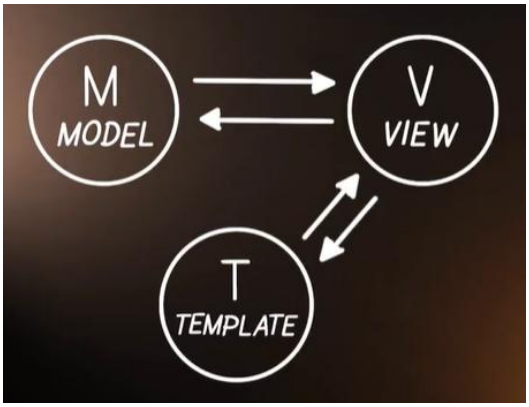


- **django-admin --help**: El comando **django-admin** no solamente sirve para crear el proyecto con el comando **startproject**, sino para muchas cosas más, para poder ver todas las cosas que hace debemos ejecutar su comando **--help**.
- **python manage.py --help**: El comando **python manage.py** no solamente sirve para ejecutar el proyecto con el método **runserver**, sino para muchas cosas más, para poder ver todas las cosas que hace debemos ejecutar su comando **--help**.

Arquitectura MVT (Model View Template)

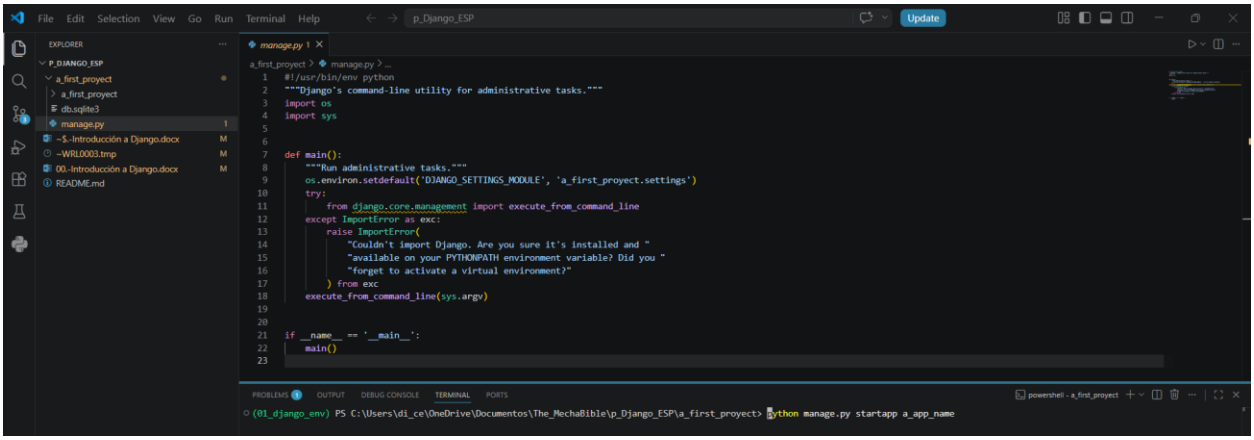
La arquitectura MVT (Model View Template) es la utilizada dentro del framework de Django, esa está pensada para

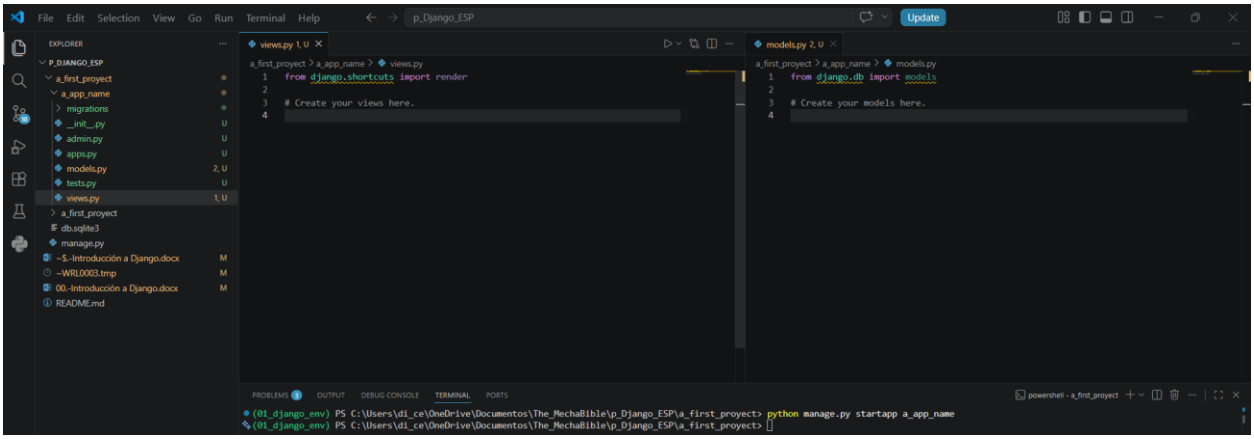
- **Model (M):** Esta es la capa de datos de la arquitectura, que indica como se guardan los datos, donde se procesan, que tipo de datos son, donde va la lógica de negocio, etc.
    - **→:** El **modelo** entrega estructuras de datos del backend hacia la vista.
    - **←:** El **modelo** recibe una petición de la **vista** y entrega hacia el **template** el dato que está buscando.
- **View (V):** Esta se refiere a un conector entre el modelo que tiene acceso a los datos y el template (UI) que los está pidiendo para saber si los puede proporcionar o no, esta parte es la encargada de controlar el flujo de peticiones SQL de la interfaz gráfica hacia la base de datos y respuestas del backend hacia el frontend.
    - **→:** La **vista** entrega al **template** un array de datos recibidos del modelo en un tipo de estructura llamada contexto, que es un diccionario de Python con información cambiante de la base de datos, para que se pueda mostrar al usuario en la UI (user interface).
    - **←:** La **vista** recibe una petición (id, request y query) del template del **template** y se conectará al **modelo** para encontrar el dato.
- **Template (T):** Se refiere a la interfaz gráfica de la aplicación, ósea donde se visualizan los datos del modelo.
    - **→:** El **template** recibe la estructura de la **vista** y la muestra al usuario.
    - **←:** El **template** entrega una petición a la **vista**, para buscar un dato en el modelo, indicando el usuario (id asignado al user en la base de datos), el request (método HTTP hecho a la API) y la query (petición SQL hacia la base de datos).



Nota: Aunque se puede hacer una conexión directa entre el modelo y el template, no es de buenas prácticas, ya que la vista realiza funcionalidades clave en la arquitectura como comprobar los permisos del usuario (autenticación), roles, etc.

- **python manage.py startapp** *a_app_name*: El método **startapp** sirve para crear una aplicación dentro de nuestro proyecto previamente creado con el comando **django-admin startproject** *a_proyect_name*, al ejecutar este comando se creará una carpeta con el nombre *a_app_name*, que contendrá varios archivos, incluidos los scripts de **models.py** y **views.py** de la arquitectura MVT, para esto debemos recordar que ya debemos haber encendido nuestro entorno virtual con el comando **&** *$HOME/.envs/01_django_env***/Scripts/Activate.ps1** dentro de la carpeta del proyecto *a_proyect_name*.



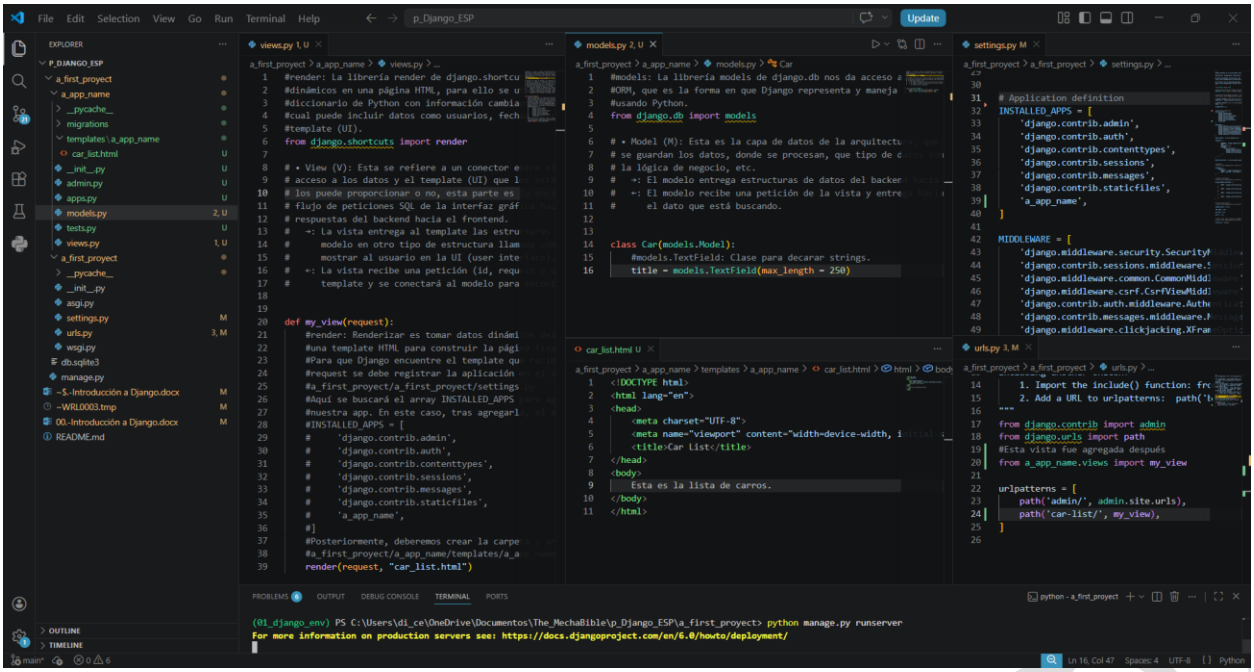


  - **models.py:** Una vez que el proyecto y la app han sido creadas, dentro del script **models.py** deberemos crear las clases que reflejen las tablas de las bases de datos que se crearán y manejarán en la operación, esto se hace a través de la librería models de django.db a través de **ORM (Object-Relational Mapping)**, que es la forma en que Django representa y maneja la base de datos usando Python.
  - **views.py:** En el script **views.py** se crearán los renders, que cargan datos dinámicos en una página HTML, para ello se utiliza un contexto, que es un diccionario de Python con información cambiante de la base de datos, la cual puede incluir datos como usuarios, fechas, etc. y los inserta en el **template** (UI).

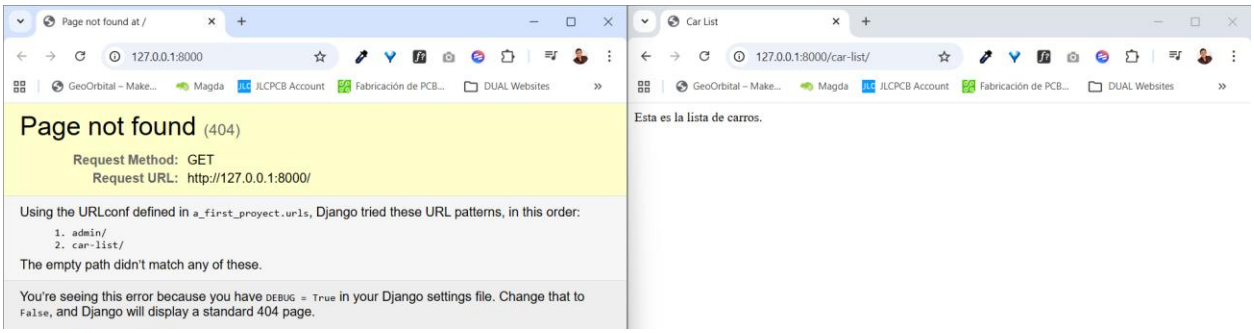
- **settings.py:** Ya habiendo creado una función dentro de **views.py** que reciba un request y lo asigne a un render, este se deberá asignar a un template, para ello se debe registrar la aplicación en el archivo *a_first_proyect*/*a_first_proyect*/**settings.py,** aquí se buscará el array INSTALLED_APPS para agregar el nombre de nuestra app. En este caso, tras agregarla, el array quedará así:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'a_app_name',
]
```

- Posteriormente, deberemos crear la carpeta y archivo del template: *a_first_proyect*/*a_app_name*/**templates/***a_app_name*/**template_1.html**, luego colocar un return en la función del render y actualizar el path de este archivo, solamente añadiendo lo que diga después de **templates/**.
- **urls.py:** Finalmente, para que podamos acceder a esa vista desde el proyecto, se debe agregar un nuevo path que haga referencia a la función que declaró el render de nuestra app en el script **views.py** y se ejecutará el comando **python manage.py runserver** para verla en el navegador.



La página principal mostrará un error 404 ya que no hemos definido la página de inicio, pero si en el localhost colocamos el endpoint del path que acabamos de crear en el script de **urls.py**, aparecerá el **template** que acabamos de crear.

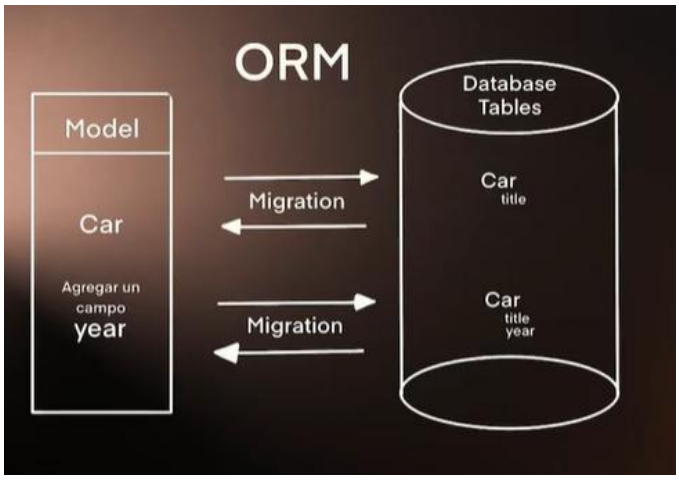


Model: ORM (Object-Relational Mapping) y Migraciones

La parte del Modelo en la arquitectura MVT se refiere a la conexión del código con la base de datos, en Django esto se maneja a través de un concepto llamado ORM (Object-Relational Mapping), que nos permite crear clases de Python que se relacionarán con tablas de la base de datos que elijamos, para no tener que escribir secuencias SQL y solo trabajar con el lenguaje Python, donde:

Clase Script = Tabla DB.

Para poder crear el contenido de una clase de Python dentro de una tabla SQL Django nos proporciona una herramienta llamada migraciones, estas nos permiten hacer cambios en la base de datos, ya sea para realizar desde Python un cambio en la database, o desde la base de datos, un cambio en la clase de Python, para que ambas estén sincronizadas.



Referencias

Platzi, Luis Martínez, “Curso de Django”, 2024 [Online], Available: https://platzi.com/cursos/django/que-es-django/